# Analiza algorytmów: Programowanie Dynamiczne i Zachłanne (Rod Cutting, LCS, Activity Selector, Huffman)

## 1 Wprowadzenie

W sprawozdaniu przedstawiono implementację i analizę wydajności algorytmów rozwiązujących cztery klasyczne problemy optymalizacyjne w zależności od różnych podejść (naiwnych, zachłannych, dynamicznych).

## 2 Metodyka pomiarów

Pomiary czasu wykonania zaimplementowano w języku C++ przy użyciu biblioteki `chrono`. Dla każdego algorytmu przeprowadzono serię testów na losowych danych wejściowych o różnym rozmiarze $n$. Wyniki uśredniono, aby zminimalizować błąd pomiarowy.

## 3 Implementacja i Analiza

### 3.1 Cięcie pręta (Rod Cutting)

Problem polega na znalezieniu sposobu pocięcia pręta o długości $n$, aby zmaksymalizować zysk, mając dany cennik.

#### 3.1.1 Implementacja

Zaimplementowano trzy wersje algorytmu:

1. **Naiwna (rekurencyjna):** $O(2^n)$ – sprawdza wszystkie możliwe podziały.
2. **Memoized (rekurencja ze spamiętywaniem):** $O(n^2)$ – zapamiętuje wyniki podproblemów.
3. **Iteracyjna (Bottom-Up):** $O(n^2)$ – wypełnia tablicę od najmniejszych podproblemów.

Listing 1: Fragment implementacji: Rod Cutting (wersja iteracyjna)

```cpp
    // Algorytm Bottom-Up o zlozonosci O(n^2).
    // Rozwiazuje podproblemy od najmniejszej dlugosci preta (j) do n.
    Solution cut_rod_bottom_up(const vector<int>& p, int n) {
      vector<int> revenue(n + 1); // Tablica zyskow
      vector<int> sol(n + 1); // Tablica rekonstrukcji rozwiazania

      revenue[0] = 0;

      // Iterujemy po dlugosciach preta od 1 do n
      for (int j = 1; j <= n; ++j) {
        int q = INT_MIN;
        // Sprawdzamy kazde mozliwe pierwsze ciecie 'i' (od 1 do j)
        for (int i = 1; i <= j; ++i) {
          // p[i-1] to cena kawalka o dlugosci i
          // revenue[j-i] to optymalny zysk dla pozostalej czesci
          int val = p[i - 1] + revenue[j - i];

          if (q < val) {
            q = val;
            sol[j] = i; // Zapamietujemy optymalne ciecie dla dlugosci j
          }
        }
        revenue[j] = q;
      }

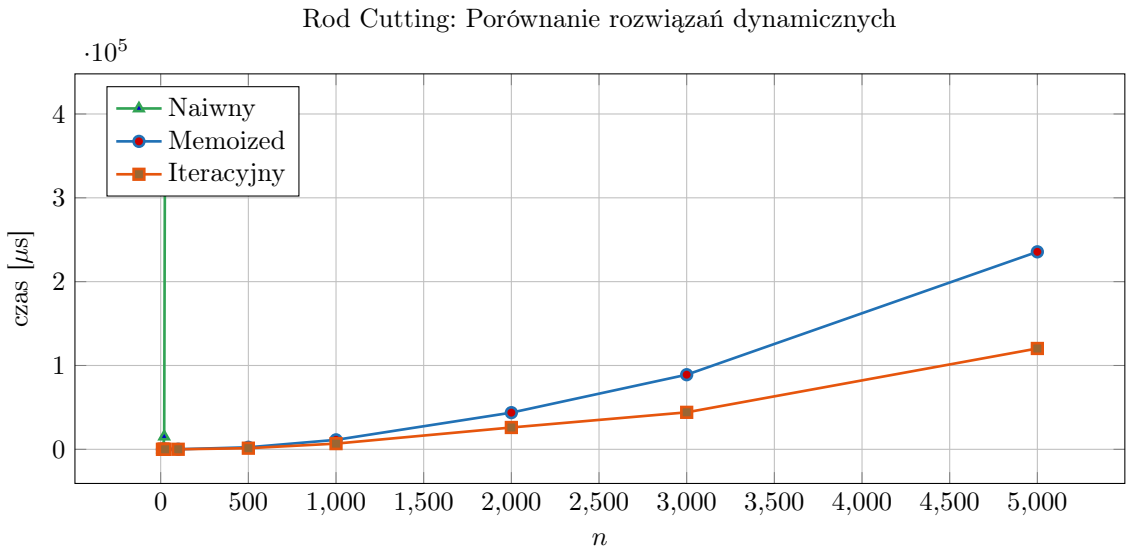
      return {revenue[n], sol};
    }
```

### 3.1.2 Wyniki pomiarów

Poniższa tabela oraz wykres przedstawiają czasy wykonania (w mikrosekundach) dla różnych długości pręta.

Tabela 1: Wyniki pomiarów dla algorytmów Rod Cutting

| $n$ | Naiwny [$\mu$s] | Memoized [$\mu$s] | Iteracyjny [$\mu$s] |
|---|---|---|---|
| 10 | 11 | 2 | 1 |
| 15 | 358 | 3 | 2 |
| 20 | 15177 | 13 | 6 |
| 25 | 407131 | 9 | 5 |
| 100 | | 94 | 54 |
| 500 | | 2414 | 1340 |
| 1000 | | 11293 | 6804 |
| 2000 | | 43746 | 26049 |
| 3000 | | 89005 | 44123 |
| 5000 | | 235576 | 120291 |




Rysunek 1: Dla dużych $n$ obie wersje dynamiczne działają w czasie wielomianowym, podczas gdy czas wersji naiwnej (widoczny tylko dla małych $n$) rośnie wykładniczo.

## 3.2 Najdłuższy Wspólny Podciąg (LCS)

Problem LCS polega na znalezieniu najdłuższego ciągu znaków występującego w obu porównywanych napisach w tej samej kolejności (niekoniecznie obok siebie).

### 3.2.1 Implementacja

Porównano podejście rekurencyjne ze spamiętywaniem oraz iteracyjne.

Listing 2: Fragment implementacji: LCS (wersja iteracyjna)

```
    // Algorytm Bottom-Up o zlozonosci O(m*n).
    // Wypelnia tablice dp, gdzie dp[i][j] to dlugosc LCS dla X[0..i-1] i Y[0..j-1].
    int run_iterative() {
      // Inicjalizacja zerami
      dp_table.assign(m + 1, vector<int>(n + 1, 0));

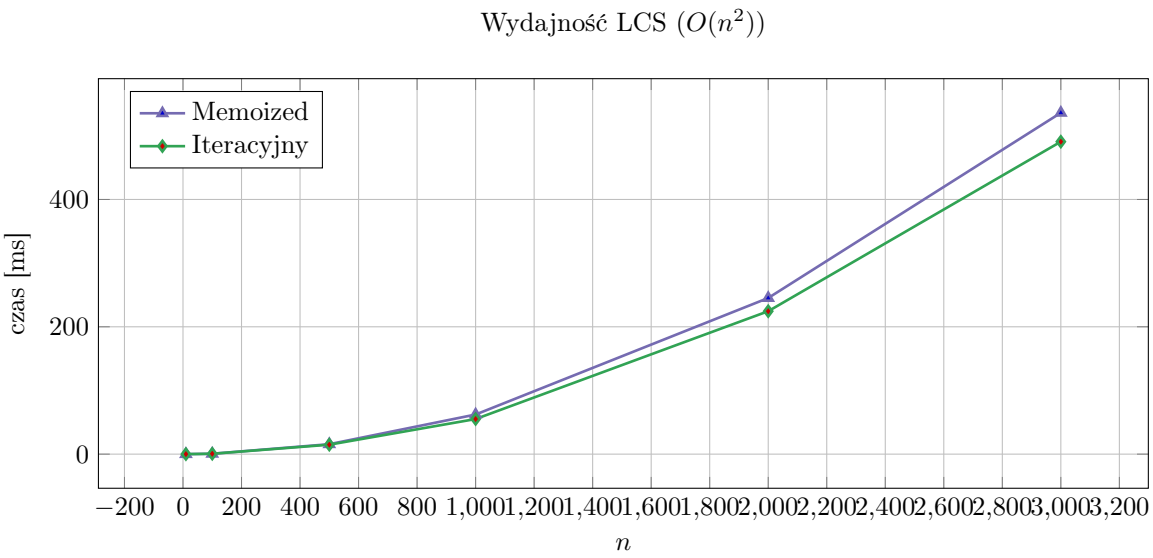
      for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
          // Jesli znaki sa takie same, wydluzamy podciag znaleziony wczesniej (po
              skosie)
          if (X[i - 1] == Y[j - 1]) {
            dp_table[i][j] = 1 + dp_table[i - 1][j - 1];
          }
          // W przeciwnym razie bierzemy lepszy wynik z sasiadow (gora lub lewo)
          else {
            dp_table[i][j] = max(dp_table[i - 1][j], dp_table[i][j - 1]);
          }
        }
      }
      table_ready = true;
      return dp_table[m][n];
    }
```

### 3.2.2 Wyniki pomiarów

Testy przeprowadzono na losowych ciągach znaków (DNA). Jednostka czasu: milisekundy.

Tabela 2: Porównanie czasów dla LCS

| $n$ | Memoized | Iteracyjny |
|---|---|---|
| 10 | 0.007768 | 0.006852 |
| 100 | 0.584897 | 0.569766 |
| 500 | 15.6002 | 14.8374 |
| 1000 | 62.1987 | 55.1179 |
| 2000 | 245.237 | 224.481 |
| 3000 | 536.111 | 490.78 |




Rysunek 2: Wykres potwierdza kwadratową złożoność obliczeniową.

## 3.3 Wybór zajęć (Activity Selector)

Problem polega na wybraniu maksymalnego podzbioru niekonfliktowych zajęć.

### 3.3.1 Implementacja

Porównano:

- **Greedy (Zachłanny):** Złożoność zdominowana przez sortowanie $O(n \log n)$. Testowano wersję standardową (sortowanie po czasie zakończenia) i zmodyfikowaną (sortowanie po czasie startu).
- **DP (Dynamiczne):** Złożoność $O(n^2)$, sprawdza wszystkie możliwe podzbiory w sposób usystematyzowany.

Listing 3: Zmodyfikowany greedy

```
    // Algorytm zachlanny dla danych posortowanych po czasie startu.
    // Zlozonosc: O(N log N) (przez sortowanie), sama selekcja O(N).
    void solve_greedy_start_sorted(vector<Activity> acts) {
      // 1. Sortowanie rosnaco po czasie ROZPOCZECIA
      sort(acts.begin(), acts.end(), [](const Activity& a, const Activity& b) {
        return a.start < b.start;
      });

      if (acts.empty()) return;
      int n = acts.size();

      // Strategia: Wybieramy zajecie o najpozniejszym starcie (ostatnie w
          posortowanym wektorze)
      // i idziemy "w tyl", szukajac kompatybilnych poprzednikow.
      vector<Activity> result;
      result.push_back(acts[n - 1]); // Pierwszy wybor (od konca)
      int last_added_index = n - 1;

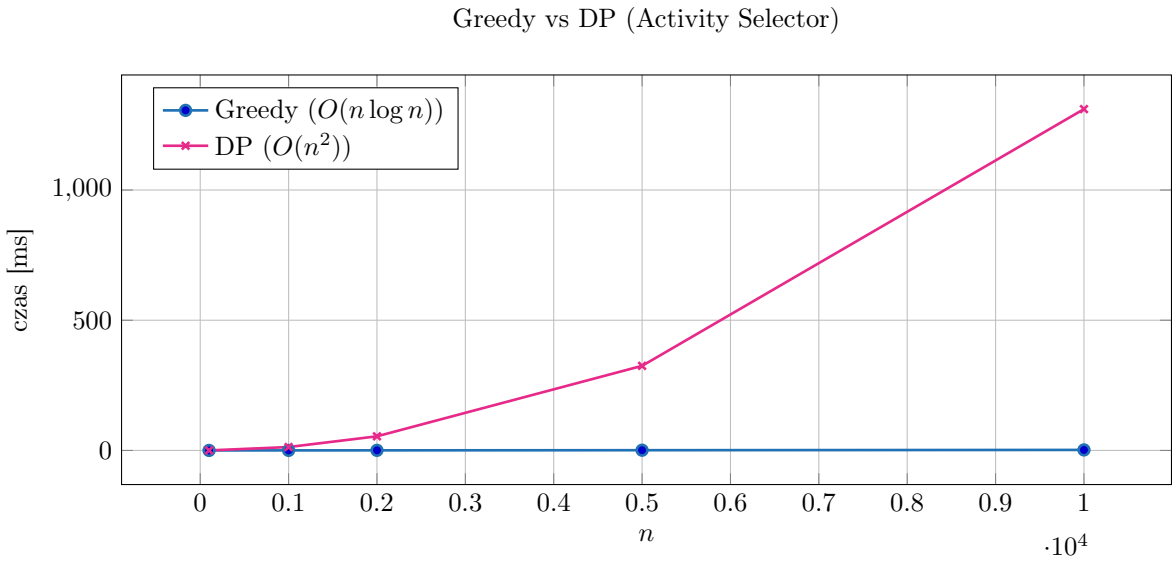
      // Iteracja wsteczna
      for (int i = n - 2; i >= 0; --i) {
        // Warunek: koniec aktualnego musi byc <= start poprzednio wybranego
        if (acts[i].finish <= acts[last_added_index].start) {
          result.push_back(acts[i]);
          last_added_index = i;
        }
      }
      // Wynik jest poprawny co do ilosci i zbioru, ale w odwrotnej kolejnosci
          chronologicznej
    }
```

### 3.3.2 Wyniki pomiarów

Dla dużych $n$ spodziewana jest drastyczna różnica między podejściem $O(n \log n)$ a $O(n^2)$.

Tabela 3: Porównanie algorytmów Greedy i DP

| $n$ | Greedy Iter [ms] | DP [ms] |
|---|---|---|
| 100 | 0.012363 | 0.127582 |
| 1000 | 0.138488 | 12.9839 |
| 2000 | 0.302775 | 54.1665 |
| 5000 | 0.822413 | 324.828 |
| 10000 | 1.9411 | 1310.76 |




Rysunek 3: Podejście zachłanne deklasuje programowanie dynamiczne dla tego problemu. Krzywa DP rośnie parabolicznie, podczas gdy Greedy pozostaje blisko osi poziomej.

## 3.4 Kodowanie Huffmana

Zaimplementowano algorytm Huffmana w dwóch wariantach:

1. **Binarny:** Standardowy, używa alfabetu $\{0, 1\}$.
2. **Ternarny:** Zmodyfikowany, używa alfabetu $\{0, 1, 2\}$. Wymaga, aby liczba liści $n$ spełniała warunek $(n-1) \pmod 2 \equiv 0$, w przeciwnym razie dodawane są węzły "dummy".

### 3.4.1 Analiza efektywności kompresji na podstawie danych eksperymentalnych

Dla tekstu wejściowego "Lorem ipsum..." (491 znaków) wygenerowano drzewa Huffmana w dwóch wariantach: binarnym i ternarnym. Poniższa tabela przedstawia zestawienie wygenerowanych kodów dla wybranych znaków.

Kody ternarne są systematycznie krótsze od binarnych. Na przykład dla spacji kod skraca się z 3 bitów do 2 trytów, a dla rzadkiego znaku 'Q' z 9 bitów do 5 trytów.

Tabela 4: Bezpośrednie porównanie kodów binarnych i ternarnych (wybrane znaki)

| **Znak** | **Kod Binarny (0-1)** | **Kod Ternarny (0-2)** | **Zysk długości** |
|---|---|---|---|
| ' ' (spacja) | `101` (3) | `20` (2) | -1 |
| 'e' | `001` (3) | `11` (2) | -1 |
| 'a' | `0101` (4) | `212` (3) | -1 |
| 's' | `1100` (4) | `00` (2) | -2 |
| 'p' | `111101` (6) | `2222` (4) | -2 |
| 'L' | `100111010` (9) | `02101` (5) | -4 |
| 'Q' | `100111011` (9) | `02102` (5) | -4 |

Całkowite długości zakodowanych ciągów dla całego tekstu przedstawiają się następująco:

Tabela 5: Sumaryczna długość zakodowanego tekstu

| **Metoda** | **Całkowita długość** |
|---|---|
| Huffman Binarny | 2058 bitów |
| Huffman Ternarny | **1311 trytów** |

Stosunek długości wyniósł:

$$\text{Ratio} = \frac{1311}{2058} \approx 0.637$$

**Wnioski:** Wynik eksperymentalny (0.637) potwierdza teorię informacji. Jeden tryt przenosi więcej informacji niż bit ($\log_2 3 \approx 1.585$ bita). Teoretycznie idealna kompresja pozwoliłaby skrócić zapis do ok. 63.1% oryginalnej długości wyrażonej w bitach. Uzyskany wynik jest bardzo bliski tej granicy.

# 4 Wnioski końcowe

Przeprowadzone eksperymenty potwierdziły, że dobór odpowiedniego algorytmu jest kluczowy dla wydajności systemu.

- W problemach takich jak *Rod Cutting* czy *LCS*, zastosowanie programowania dynamicznego pozwala rozwiązywać problemy w czasie wielomianowym, które naiwnie wymagałyby czasu wykładniczego.
- W przypadku *Activity Selector*, istnienie własności wyboru zachłannego pozwala zredukować złożoność do $O(n \log n)$, co czyni użycie DP ($O(n^2)$) nieefektywnym.
- Dla kodowania Huffmana, zwiększenie podstawy systemu kodowania skraca długość wynikowego napisu, co może być korzystne w przesyłaniu danych.